

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 18
“MESIN ABSTRAK”



DISUSUN OLEH:
MUHAMMAD NABIL RAISSA P
109082500127
S1IF-13-02

DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

18.1 Pengantar Mesin Abstrak Model komputasi terdiri dari data yang dapat diolah dan operasi-operasi dasar pengolahan data tersebut. Mesin abstrak adalah model komputasi yang dirancang di atas model mesin komputasi yang telah ada, yaitu tipe data dan operasi-operasi dasarnya dibuat menggunakan tipe data dan operasi-operasi yang tersedia di mesin di bawahnya. Teknik ini merupakan salah satu cara untuk membangun perangkat lunak.

18.2 Contoh Kasus Mesin Domino Tipe data dan operasi dasar mesin domino:

1) Mesin abstrak ini menyediakan operasi-operasi dasar yang dapat dilakukan untuk bermain domino; seperti:

- kocok kartu; mengubah dan mengacak urutan kartu yang diberikan, sehingga urutannya tidak dapat diduga lagi.
- ambil kartu; mengambil satu kartu dari tumpukan kartu.
- gambar kartu; melihat gambar (suit) kartu beserta nilainya.
- nilai kartu; memberikan nilai kartu

2) Operasi-operasi tersebut bekerja terhadap data yang spesifik, yaitu kartu domino

- Setiap kartu mempunyai dua sisi, yang masing-masing mempunyai titik/pip dari nol (tidak ada) s.d. maksimum 6 pip.
- Jika satu sisi kartu dianggap gambar (suit), maka sisi lainnya dianggap nilai dari kartu dengan gambar (suit) tersebut. Contohnya kartu (1,4) dapat dianggap kartu gambar 1 (titik) dengan nilai 4 atau kartu gambar 4 (titik) dengan nilai 1.
- Nilai dari setiap kartu, yaitu jumlah titik dari kedua sisi, dari nol s.d. 12 poin
- Karena tidak ada duplikat, kartu (1,4) sama saja dengan kartu (4,1), maka semuanya ada 28 kartu yang berbeda.

Kasus sederhana bermain kartu domino:

- Diberikan sebuah kartu, cari kartu lain dari tumpukan kartu yang salah satu gambar (suit) nya sama dengan kartu pertama.

- Uji apakah kartu yang dipegang adalah kartu balak, yaitu kartu yang kedua sisinya mempunyai nilai yang sama.

- Diberikan sepasang kartu, apakah nilai kedua kartu tersebut 12?

Beberapa permainan kartu domino:

- Gapleh; membuat rangkaian kartu domino, yang sambung menyambung berdasarkan kesamaan gambar (suit)

- Kiu-kiu; mendapatkan 4 kartu domino dengan total nilai yang paling tinggi, sesuai dengan aturan permainan.

- Luzon; sesuai dengan aturan permainan, mencari sebanyak-banyaknya pasangan kartu domino dengan jumlah nilai 12.

- Texas 42; seperti permainan cangkulan dengan menggunakan kartu domino.

SOAL LATIHAN

Statement perulangan

1.

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

type Domino struct {
    sisi1 int
    sisi2 int
    nilai int
    isBalak bool
}

type Dominoes struct {
    kartu [28]Domino
    tersisa int
}

func kocokKartu(d *Dominoes) {
    for i := 0; i < d.tersisa; i++ {
        j := rand.Intn(d.tersisa)
        d.kartu[i], d.kartu[j] = d.kartu[j], d.kartu[i]
    }
}

func ambilKartu(d *Dominoes) Domino {
    if d.tersisa == 0 {
        return Domino{}
    }
    d.tersisa--
    kartuDiambil := d.kartu[d.tersisa]
    return kartuDiambil
}

func gambarKartu(d Domino, suit int) int {
    if suit == 1 {
        return d.sisi1
    }
    return d.sisi2
}

func nilaiKartu(d Domino) int {
```

```

    return d.nilai
}

func main() {
    var dek Dominoes
    dek.tersisa = 28

    idx := 0
    for i := 0; i <= 6; i++ {
        for j := i; j <= 6; j++ {
            dek.kartu[idx] = Domino{
                sisi1: i,
                sisi2: j,
                nilai: i + j,
                isBalak: i == j,
            }
            idx++
        }
    }

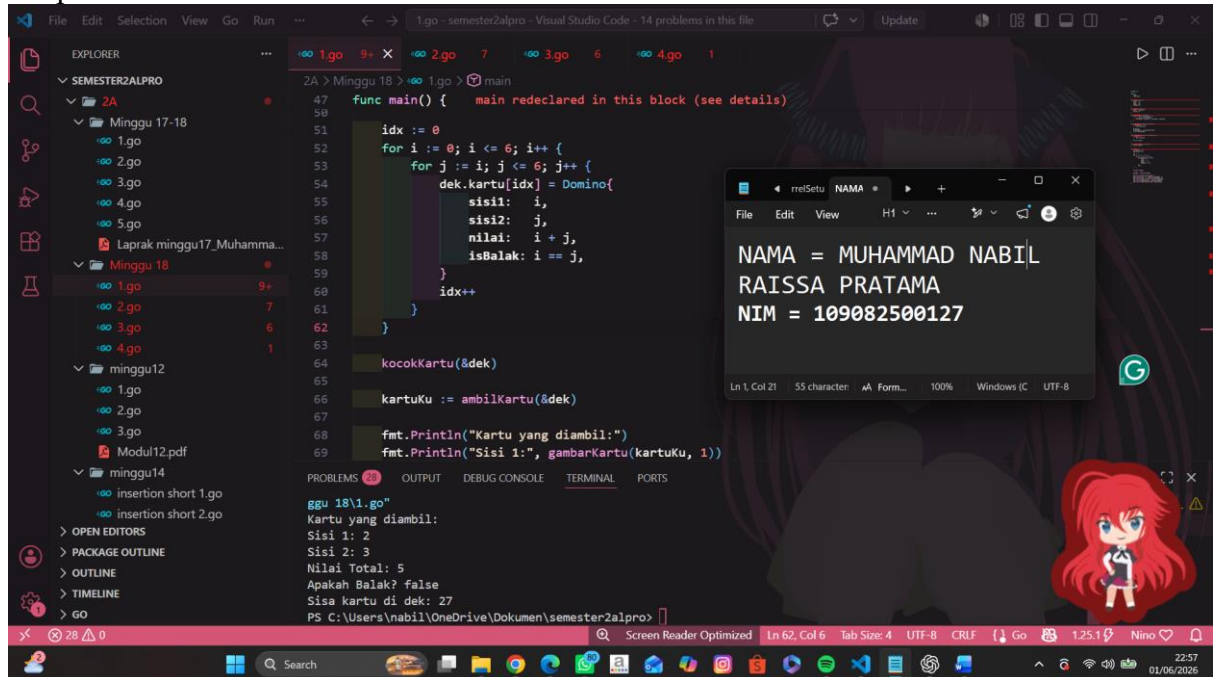
    kocokKartu(&dek)

    kartuKu := ambilKartu(&dek)

    fmt.Println("Kartu yang diambil:")
    fmt.Println("Sisi 1:", gambarKartu(kartuKu, 1))
    fmt.Println("Sisi 2:", gambarKartu(kartuKu, 2))
    fmt.Println("Nilai Total:", nilaiKartu(kartuKu))
    fmt.Println("Apakah Balak?", kartuKu.isBalak)
    fmt.Println("Sisa kartu di dek:", dek.tersisa)
}

```

Output:



The screenshot shows a Visual Studio Code editor with a Go file named `1.go` in the `2A` folder. The code defines a `main` function that uses a `Domino` struct and a `Dominoes` slice. It includes a `kocokKartu` function and a `kartuKu` variable. The output window shows the following text:

```
NAMA = MUHAMMAD NABIL  
RAISSA PRATAMA  
NIM = 109082500127
```

The terminal window shows the following output:

```
ggu 18\1.go  
Kartu yang diambil:  
Sisi 1: 2  
Sisi 2: 3  
Nilai Total: 5  
Apakah Balak? false  
Sisa kartu di dek: 27  
PS C:\Users\nabil\OneDrive\Dokumen\semester2alpro>
```

Deskripsi Program:

2.

Source Code:

```
package main  
  
import (  
    "fmt"  
    "math/rand"  
)  
  
type Domino struct {  
    sisi1 int  
    sisi2 int  
    nilai int  
    isBalak bool  
}  
  
type Dominoes struct {  
    kartu [28]Domino  
    tersisa int  
}  
  
func kocokKartu(d *Dominoes) {  
    for i := 0; i < d.tersisa; i++ {
```

```

        j := rand.Intn(d.tersisa)
        d.kartu[i], d.kartu[j] = d.kartu[j], d.kartu[i]
    }
}

func ambilKartu(d *Dominoes) Domino {
    if d.tersisa == 0 {
        return Domino{}
    }
    d.tersisa--
    return d.kartu[d.tersisa]
}

func gambarKartu(d Domino, suit int) int {
    if suit == 1 {
        return d.sisi1
    }
    return d.sisi2
}

func nilaiKartu(d Domino) int {
    return d.nilai
}

func galiKartu(d *Dominoes, target Domino) {
    for d.tersisa > 0 {
        k := ambilKartu(d)
        if gambarKartu(k, 1) == gambarKartu(target, 1) ||
            gambarKartu(k, 1) == gambarKartu(target, 2) ||
            gambarKartu(k, 2) == gambarKartu(target, 1) ||
            gambarKartu(k, 2) == gambarKartu(target, 2) {
            fmt.Printf("Dapet nih yang nyambung! [%d|%d]\n", gambarKartu(k, 1),
gambarKartu(k, 2))
            return
        }
        fmt.Printf("Buang kartu: [%d|%d] (Gak nyambung)\n", gambarKartu(k, 1),
gambarKartu(k, 2))
    }
    fmt.Println("Yah, tumpukan abis, ga ada yang nyambung.")
}

func sepasangKartu(k1 Domino, k2 Domino) bool {
    return nilaiKartu(k1)+nilaiKartu(k2) == 12
}

func main() {
    var dek Dominoes
    dek.tersisa = 28
    idx := 0
    for i := 0; i <= 6; i++ {

```

```

    for j := i; j <= 6; j++ {
        dek.kartu[idx] = Domino{
            sisi1: i,
            sisi2: j,
            nilai: i + j,
            isBalak: i == j,
        }
        idx++
    }
}

kocokKartu(&dek)

targetKartu := ambilKartu(&dek)
fmt.Printf("Kartu patokan di meja: [%d|%d]\n", gambarKartu(targetKartu, 1),
gambarKartu(targetKartu, 2))
fmt.Println("Mulai gali tumpukan...")
galiKartu(&dek, targetKartu)

kartuA := Domino{sisi1: 3, sisi2: 4, nilai: 7, isBalak: false}
kartuB := Domino{sisi1: 2, sisi2: 3, nilai: 5, isBalak: false}
fmt.Printf("\nCek sepasang kartu A[%d|%d] dan B[%d|%d], totalnya 12 ga? %v\n",
gambarKartu(kartuA, 1), gambarKartu(kartuA, 2), gambarKartu(kartuB, 1),
gambarKartu(kartuB, 2), sepasangKartu(kartuA, kartuB))
}

```

Output:

The screenshot shows a Visual Studio Code editor with a Go file named `2A.go`. The code defines a `Domino` struct with fields `sisi1`, `sisi2`, `nilai`, and `isBalak`. It includes functions for dealing cards (`kocokKartu`), checking a pair of cards (`sepasangKartu`), and a main function that deals a target card and searches for a pair. The output window shows the following results:

```

NAMA = MUHAMMAD NABIL
RAISSA PRATAMA
NIM = 109082500127

```

The status bar at the bottom indicates the file is at line 81, column 21, with a tab size of 4, UTF-8 encoding, and CRLF line endings. The system tray shows the date and time as 22:58 on 01/06/2026.

Deskripsi Program:

3.

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

type Domino struct {
    sisi1 int
    sisi2 int
    nilai int
    isBalak bool
}

type Dominoes struct {
    kartu [28]Domino
    tersisa int
}

func kocokKartu(d *Dominoes) {
    rand.Seed(time.Now().UnixNano())
    for i := 0; i < d.tersisa; i++ {
        j := rand.Intn(d.tersisa)
        d.kartu[i], d.kartu[j] = d.kartu[j], d.kartu[i]
    }
}

func ambilKartu(d *Dominoes) Domino {
    if d.tersisa == 0 {
        return Domino{}
    }
    d.tersisa--
    return d.kartu[d.tersisa]
}

func gambarKartu(d Domino, suit int) int {
    if suit == 1 {
        return d.sisi1
    }
    return d.sisi2
}
```

```

type Pemain struct {
    nama string
    tangan []Domino
}

func main() {
    var dek Dominoes
    dek.tersisa = 28
    idx := 0
    for i := 0; i <= 6; i++ {
        for j := i; j <= 6; j++ {
            dek.kartu[idx] = Domino{sisi1: i, sisi2: j, nilai: i + j, isBalak: i == j}
            idx++
        }
    }
    kocokKartu(&dek)

    pemain := []Pemain{
        {nama: "Pemain 1"},
        {nama: "Pemain 2"},
        {nama: "Pemain 3"},
        {nama: "Pemain 4"},
    }

    for i := 0; i < 7; i++ {
        for p := 0; p < 4; p++ {
            pemain[p].tangan = append(pemain[p].tangan, ambilKartu(&dek))
        }
    }

    fmt.Println("=== GAME GAPLEH DIMULAI ===")

    ujungKiri := -1
    ujungKanan := -1
    giliran := 0
    passCount := 0
    gameSelesai := false

    for !gameSelesai {
        p := &pemain[giliran]
        kartuDimainkan := false

        for i, k := range p.tangan {
            sisi1 := gambarKartu(k, 1)
            sisi2 := gambarKartu(k, 2)

            if ujungKiri == -1 && ujungKanan == -1 {
                ujungKiri = sisi1
                ujungKanan = sisi2
                fmt.Printf("%s mulai dengan kartu: [%d|%d]\n", p.nama, sisi1, sisi2)
            }
        }
    }
}

```

```

        kartuDimainkan = true
    } else {
        if sisi1 == ujungKiri {
            ujungKiri = sisi2
            fmt.Printf("%s mainin [%d|%d] di kiri. Ujung sekarang: %d dan %d\n",
p.nama, sisi1, sisi2, ujungKiri, ujungKanan)
            kartuDimainkan = true
        } else if sisi2 == ujungKiri {
            ujungKiri = sisi1
            fmt.Printf("%s mainin [%d|%d] di kiri. Ujung sekarang: %d dan %d\n",
p.nama, sisi1, sisi2, ujungKiri, ujungKanan)
            kartuDimainkan = true
        } else if sisi1 == ujungKanan {
            ujungKanan = sisi2
            fmt.Printf("%s mainin [%d|%d] di kanan. Ujung sekarang: %d dan
%d\n", p.nama, sisi1, sisi2, ujungKiri, ujungKanan)
            kartuDimainkan = true
        } else if sisi2 == ujungKanan {
            ujungKanan = sisi1
            fmt.Printf("%s mainin [%d|%d] di kanan. Ujung sekarang: %d dan
%d\n", p.nama, sisi1, sisi2, ujungKiri, ujungKanan)
            kartuDimainkan = true
        }
    }
}

if kartuDimainkan {
    p.tangan = append(p.tangan[:i], p.tangan[i+1:]...)
    passCount = 0
    break
}

if !kartuDimainkan {
    fmt.Printf("%s PASS (Gak ada kartu yang nyambung)\n", p.nama)
    passCount++
}

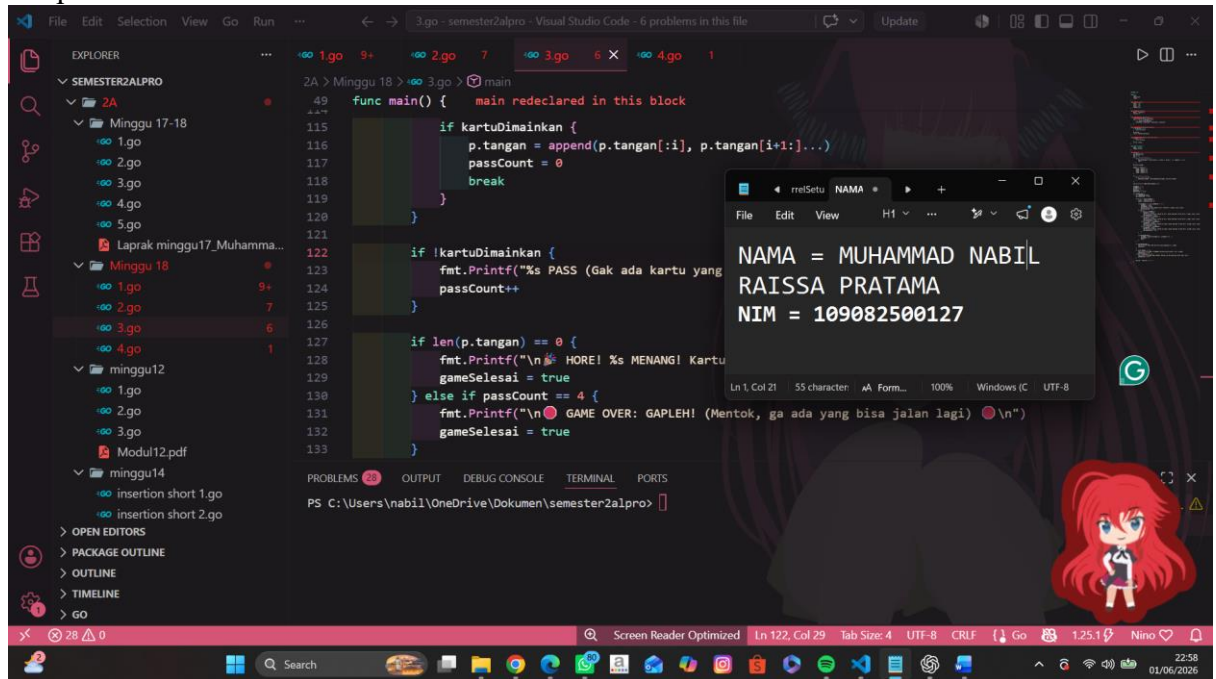
if len(p.tangan) == 0 {
    fmt.Printf("\n🀄 HORE! %s MENANG! Kartunya habis duluan! 🀄\n",
p.nama)
    gameSelesai = true
} else if passCount == 4 {
    fmt.Printf("\n🀇 GAME OVER: GAPLEH! (Mentok, ga ada yang bisa jalan
lagi) 🀇\n")
    gameSelesai = true
}

giliran = (giliran + 1) % 4
}

```

```
}
```

Output:



Deskripsi Program:

4.

Source Code:

```
package main  
  
import (  
    "bufio"  
    "fmt"  
    "os"  
    "strings"  
)  
  
var pita string  
var indeks int  
var currentChar byte  
var isEOP bool  
  
func start(input string) {  
    pita = input  
    indeks = 0
```

```

    if len(pita) > 0 {
        currentChar = pita[indeks]
        isEOP = (currentChar == '.')
    } else {
        isEOP = true
    }
}

func maju() {
    if indeks < len(pita)-1 {
        indeks++
        currentChar = pita[indeks]
        isEOP = (currentChar == '.')
    } else {
        isEOP = true
    }
}

func eop() bool {
    return isEOP
}

func cc() byte {
    return currentChar
}

func main() {
    fmt.Print("Masukan teks (akhiri dengan titik): ")
    scanner := bufio.NewScanner(os.Stdin)
    scanner.Scan()
    input := scanner.Text()

    input = strings.ToUpper(input)

    start(input)

    totalKarakter := 0
    jumlahA := 0
    jumlahLE := 0
    var prevChar byte

    fmt.Print("Karakter yang terbaca: ")
    for !eop() {
        c := cc()
        fmt.Printf("%c", c)

        totalKarakter++

        if c == 'A' {
            jumlahA++

```

```

    }

    if prevChar == 'L' && c == 'E' {
        jumlahLE++
    }

    prevChar = c
    maju()
}
fmt.Println()

fmt.Println("Total karakter:", totalKarakter)
fmt.Println("Jumlah huruf 'A':", jumlahA)

frekuensi := 0.0
if totalKarakter > 0 {
    frekuensi = float64(jumlahA) / float64(totalKarakter)
}
fmt.Printf("Frekuensi huruf 'A' terhadap total: %.4f\n", frekuensi)
fmt.Println("Jumlah pasangan kata 'LE':", jumlahLE)
}

```

Output:

The screenshot shows a Visual Studio Code editor with a Go file named '2A.go'. The code defines a 'main' function that takes a string 'input' and processes it. It counts the total number of characters, the number of 'A's, and the number of 'LE' pairs. The output window displays the results for the input string 'MUHAMMAD NABIL RAISSA PRATAMA'.

```

2A > Minggu 18 > 2A.go > ...
44 func main() { main redeclared in this block
49
50     input = strings.ToUpper(input)
51
52     start(input)
53
54     totalKarakter := 0
55     jumlahA := 0
56     jumlahLE := 0
57     var prevChar byte
58
59     fmt.Print("Karakter yang terbaca: ")
60     for loop() {
61         c := cc()
62         fmt.Printf("%c", c)
63
64         totalKarakter++
65
66         if c == 'A' {
67             jumlahA++
68         }
69     }
70     fmt.Println()
71     fmt.Println("Total karakter:", totalKarakter)
72     fmt.Println("Jumlah huruf 'A':", jumlahA)
73     frekuensi := 0.0
74     if totalKarakter > 0 {
75         frekuensi = float64(jumlahA) / float64(totalKarakter)
76     }
77     fmt.Printf("Frekuensi huruf 'A' terhadap total: %.4f\n", frekuensi)
78     fmt.Println("Jumlah pasangan kata 'LE':", jumlahLE)
79 }

```

Output:

```

Karakter yang terbaca: MUHAMMAD NABIL RAISSA PRATAMA
Total karakter: 28
Jumlah huruf 'A': 1
Frekuensi huruf 'A' terhadap total: 0.0357
Jumlah pasangan kata 'LE': 0

```

Deskripsi Program:

Jadi di tugas domino ini, intinya aku bikin simulasi game Gapleh komplit dari nol. Awalnya aku nyiapin 28 kartu domino standar, terus ngocok semuanya secara acak

sebelum dibagiin rata ke empat pemain yang masing-masing dapet 7 kartu. Nah, pas gamenya mulai, program bakal muter terus buat ngecek kartu di tangan tiap pemain secara bergiliran, nyari mana yang angkanya bisa nyambung sama ujung kiri atau ujung kanan di meja. Kalau ada yang cocok, kartunya langsung turun meja, tapi kalau lagi ampas dan zonk, pemainnya otomatis "PASS". Simulasi ini bakal terus jalan otomatis sampe akhirnya ada yang menang gara-gara kartunya habis duluan, atau malah *game over* gara-gara semua pemain mentok nggak bisa jalan alias Gapleh!

DAFTAR PUSTAKA